



Lecture 3

Robustness to Functional Form Assumptions

Dave Armstrong

Today's Robustness Question

What assumption/abstraction are we perturbing?

- That we have estimated the appropriate functional form of the relationship between x and y .

Why might this assumption fail?

- The estimated relationship is too restrictive.
- The estimated relationship is overly complex when a simpler one is sufficient.
- The estimated relationship imposes shape constraints that are not supported by the data.

Today's Robustness Question

What assumption/abstraction are we perturbing?

- That we have estimated the appropriate functional form of the relationship between x and y .

What kinds of inferential instability can result?

- Wrong relationship shape.
- Improper estimation of effect sizes/directions.
- Spurious thresholds, turning points, or saturation effects.
- Poor predictions.

Today's Robustness Question

What assumption/abstraction are we perturbing?

- That we have estimated the appropriate functional form of the relationship between x and y .

What tools exist to evaluate robustness?

- Dummy coding independent variables.
- Monotone regression relationships.
- Polynomials and non-linear transformations
- Regression splines
- Smoothing-splines/Generalized Additive Models

Today's Robustness Question

What assumption/abstraction are we perturbing?

- That we have estimated the appropriate functional form of the relationship between x and y .

What are the limits of these tools?

- Non-linear transformations and polynomials require arbitrary choice of power or basis dimension.
- Dummy coding independent variables is only really possible for ordinal variables with relatively few levels.
- Regression splines can be noisy.
- Smoothing splines/GAMs require smoothing parameters to be chosen either arbitrarily or tuned from the data.

Today's Robustness Question

What assumption/abstraction are we perturbing?

- That we have estimated the appropriate functional form of the relationship between x and y .

What are the limits of these tools?

- Non-linear transformations and polynomials require arbitrary choice of power or basis dimension.
- Dummy coding independent variables is only really possible for ordinal variables with relatively few levels.
- Regression splines can be noisy.
- Smoothing splines/GAMs require smoothing parameters to be chosen either arbitrarily or tuned from the data.

Data-driven functional forms may fit the current sample well but fail to generalize to new data.

Functional Form Assumptions.

When estimating a parametric (linear) model, particularly when interested in the relationship between x and y , we estimate:

$$y_i = a + f(x_i) + f(\mathbf{z}_i) + e_i$$

Where:

- x is a variable of interest
- $\mathbf{z}$ is a vector of covariates.

Generally $f(x)$ is parametric e.g.,:

- linear: $f(x) = b_1 x$
- log: $f(x) = b_1 \log(x)$
- quadratic: $f(x) = b_1 x + b_2 x^2$

Functional Form Assumptions.

When estimating a parametric (linear) model, particularly when interested in the relationship between x and y , we estimate:

$$y_i = a + f(x_i) + f(\mathbf{z}_i) + e_i$$

Where:

- x is a variable of interest
- $\mathbf{z}$ is a vector of covariates.

Generally $f(x)$ is parametric e.g.,:

- linear: $f(x) = b_1 x$
- log: $f(x) = b_1 \log(x)$
- quadratic: $f(x) = b_1 x + b_2 x^2$

The precise choice of $f(x)$ is usually guided by theory.

Functional Form Perturbations

We often want to identify how robust our functional form choices are to plausible alternatives. We can think of these from least to most complex.

- Linear and flat: $f(x) = 0x$
- Linear and not flat: $f(x) = b_1x$
- Non-linear, monotonic: $f(x) = b_1 \log(x)$ or $f(x) = b_1x^2$
- Non-linear, non-monotonic: $f(x) = b_1x + b_2x^2$
- Non-linear, flexible: $f(x)$ estimate flexibly with minimal functional form assumptions.

Functional Form Perturbations

We often want to identify how robust our functional form choices are to plausible alternatives. We can think of these from least to most complex.

- Linear and flat: $f(x) = 0x$
- Linear and not flat: $f(x) = b_1x$
- Non-linear, monotonic: $f(x) = b_1 \log(x)$ or $f(x) = b_1x^2$
- Non-linear, non-monotonic: $f(x) = b_1x + b_2x^2$
- Non-linear, flexible: $f(x)$ estimate flexibly with minimal functional form assumptions.

Robust conclusions are those that are not sensitive to choices of more complex models.

- They may be sensitive to choosing simpler models, but if those simpler models fit worse the lack of robust is expected and unproblematic.

Robustness of Functional Forms

Not all robustness failures are equally important

- If you hypothesize and fit a quadratic relationship, you expect it to be sensitive to choosing a linear or flat relationship instead.
 - These *impose additional restrictions* on the estimated relationship.
- Conversely, if you hypothesize and fit a linear relationship and find that this choice is sensitive to fitting a quadratic relationship instead.
 - This *relaxes a restriction* - the quadratic model *could have found* a linear relationship if that happened to best describe the relationships in the data.

Robustness of Functional Forms

Not all robustness failures are equally important

- If you hypothesize and fit a quadratic relationship, you expect it to be sensitive to choosing a linear or flat relationship instead.
 - These *impose additional restrictions* on the estimated relationship.
- Conversely, if you hypothesize and fit a linear relationship and find that this choice is sensitive to fitting a quadratic relationship instead.
 - This *relaxes a restriction* - the quadratic model *could have found* a linear relationship if that happened to best describe the relationships in the data.

As a general principle, sensitivity to relaxing model restrictions combined with precise evidence for the alternative is often more concerning than sensitivity to imposing additional restrictions.

Outline

We are going to proceed in a couple of different directions.

1. Robustness testing for ordinal independent variables.

- Linear effect
- Monotonic effect (relax equal spacing assumption)
- Unrestricted categorical effect (relax ordinality assumption)

Outline

We are going to proceed in a couple of different directions.

1. Robustness testing for ordinal independent variables.

- Linear effect
- Monotonic effect (relax equal spacing assumption)
- Unrestricted categorical effect (relax ordinality assumption)

2. Robustness testing for continuous independent variables.

- Linear effect
- Parametric non-linear effect (relax linearity assumption)
- shape-constrained non-linear effect (relax parametric form assumption while retaining monotonicity)
- Unrestricted non-linear effect (relax monotonicity assumption)

Outline

We are going to proceed in a couple of different directions.

1. Robustness testing for ordinal independent variables.

- Linear effect
- Monotonic effect (relax equal spacing assumption)
- Unrestricted categorical effect (relax ordinality assumption)

2. Robustness testing for continuous independent variables.

- Linear effect
- Parametric non-linear effect (relax linearity assumption)
- shape-constrained non-linear effect (relax parametric form assumption while retaining monotonicity)
- Unrestricted non-linear effect (relax monotonicity assumption)

At each stage, we evaluate whether our conclusions survive the subsequent assumption relaxation.



Linearity and Multi-Category Variables

Nominal Variables → Dummy Regressors → 😊

Linearity and Multi-Category Variables

Nominal Variables → Dummy Regressors → 😊 The waters are a bit murkier for ordinal variables (e.g., state repression or political ideology).

- These variables are often operationalized with relatively few categories.
- However, we often have a strong suspicion that the relationship between these variables and the response is "roughly linear".
- If the relationship is *not* linear and we represent it with a line, then we are getting a *biased* estimate of the relationship.
- If the relationship could be represented linearly, and we represent it with a series of dummy regressors, we are getting estimates that are *inefficient*

Robustness of the Linearity Assumption

Consider the model:

$$y = f(x) + g(\mathbf{Z}) + \varepsilon$$

Often, we want to evaluate whether a linear approximation is sufficient.

Target : $f(x) = \beta_0 + \beta_1 x$

Alternative : $f(x) \neq \beta_0 + \beta_1 x$ (i.e., the function is more complicated)

- We don't have to have know or specify the functional form of the alternative hypothesis, rather just that it is more complicated than linear.

Target and Alternative Models

Switching from treating the variable as numeric to categorical allows us to estimate the linear, monotone (retaining monotonicity while relaxing the equal-spacing assumption) and unrestricted (relaxing the monotonicity assumption) categorical models.

$$\text{Target : } f(x) = \beta_0 + \beta_1 x$$

$$\begin{aligned} \text{Monotone : } f(x) &= \beta_0 + \beta_1^* I(x = 2) + \beta_2^* I(x = 3) + \beta_3^* I(x = 4) + \beta_4^* I(x = 5) \\ \text{subject to: } &0 \leq \beta_1^* \leq \beta_2^* \leq \beta_3^* \leq \beta_4^* \end{aligned}$$

$$\text{Unrestricted : } f(x) = \beta_0 + \beta_1^* I(x = 2) + \beta_2^* I(x = 3) + \beta_3^* I(x = 4) + \beta_4^* I(x = 5)$$

where $I(\cdot)$ is an indicator function such that $I(\cdot)$ is 1 if the expression inside is true and 0 otherwise.

An Example

I generated data such that $x_i \in \{1, 2, 3, 4, 5\}$ and

$$y_i = 2 + x + \varepsilon_i$$

where $\varepsilon_i \sim N(0, 2)$.



An Example

I generated data such that $x_i \in \{1, 2, 3, 4, 5\}$ and

$$y_i = 2 + x + \varepsilon_i$$

where $\varepsilon_i \sim N(0, 2)$.

Since the effect in the alternative model is a function of multiple coefficients, we can evaluate the robustness of an effect plot.

Operationalizing Monotonicity.

We take a "pooling adjacent violators" approach to imposing monotonicity. It takes the following steps:

1. Estimate the unconstrained model.
2. Adjacent categories that violate the monotonicity constraint are pooled until a monotone relationship is obtained.
3. Repeat until all values have been evaluated.

The resulting variable will be monotonically related to the outcome.

Operationalizing Monotonicity.

We take a "pooling adjacent violators" approach to imposing monotonicity. It takes the following steps:

1. Estimate the unconstrained model.
2. Adjacent categories that violate the monotonicity constraint are pooled until a monotone relationship is obtained.
3. Repeat until all values have been evaluated.

The resulting variable will be monotonically related to the outcome.

The `make_mono_var()` function in the `nprobustness` package will do this for you.



Example Data

```
# Generate data to be used in the example
set.seed(519)
x <- rep(1:5, 100)
x <- x[order(x)]
x_fac = as.factor(x)

y <- 2 + .5*x + rnorm(500,0,5)

ex_df <- data.frame(
  x = x,
  x_fac = x_fac,
  y= y
)

# run linear model
lmod <- lm(y ~ x, data=ex_df)

# run unrestricted categorical model
umod <- lm(y ~ x_fac, data=ex_df)

# make monotone x_fac variable: x_fac_ord
ex_df <- make_mono_var(umod, ex_df,
  "x_fac", direction="up")

# run monotone model
mmod <- lm(y ~ x_fac_ord, data=ex_df)
```

```
library(marginaleffects)
library(nprobustness)
# make prediction data frame
pred_df <- ex_df |>
  summarise(n = n(),
    .by = c(x, x_fac, x_fac_ord)) |>
  mutate(x_fac = as.factor(x),
    x_fac_ord = as.factor(x_fac_ord))

# calculate predictions for each model
# note - use the name of the original numeric variable
leff <- predictions(lmod,
  by="x",
  newdata = pred_df)
meff <- predictions(mmod,
  by="x",
  newdata = pred_df)
ueff <- predictions(umod,
  by = "x",
  newdata = pred_df)

# save standard errors for each model's predictions
sd_leff <- sqrt(diag(vcov(leff)))
sd_meff <- sqrt(diag(vcov(meff)))
sd_ueff <- sqrt(diag(vcov(ueff)))
```


Calculating Robustness for Predictions

```
# robustness of linear vs monotonic predictions
rob_lm <- sapply(1:5, \(i)
  robustness(dnorm, dnorm,
    target_args = list(mean = coef(leff)[i],
                       sd = sd_leff[i]),
    alt_args = list(mean = coef(meff)[i],
                    sd = sd_meff[i])))

# robustness of linear vs unrestricted predictions
rob_lu <- sapply(1:5, \(i)
  robustness(dnorm, dnorm,
    target_args = list(mean = coef(leff)[i],
                       sd = sd_leff[i]),
    alt_args = list(mean = coef(ueff)[i],
                    sd = sd_ueff[i])))

# robustness of monotone vs unrestricted predictions
rob_mu <- sapply(1:5, \(i)
  robustness(dnorm, dnorm,
    target_args = list(mean = coef(meff)[i],
                       sd = sd_meff[i]),
    alt_args = list(mean = coef(ueff)[i],
                    sd = sd_ueff[i])))
```

The average OVL and NP statistics are reasonably high indicating considerable overlap in the effects.

rob_res

##	base	alt	ovl	np	js	
## 1	Linear	Monotone	0.7094441	0.7437994	0.12961241	0.3
## 2	Linear	Unrestricted	0.6829746	0.6937582	0.14533502	0.3
## 3	Monotone	Unrestricted	0.9277159	0.9009327	0.01842198	0.0

Calibrating Robustness Measures

Even though some of these values are bounded, we may not know what exactly to expect in any particular situation regarding the level of robustness.

What level of robustness would we expect?

- A simulation can help.

Simulation Steps

1. Estimate baseline model.
2. Use the original data, coefficients and error variance estimate to simulate $\tilde{y}$.
3. Regress $\tilde{y}$ on the linear x and the categorical $x^{(c)}$.
4. Calculate robustness for the two models.
5. Repeat R times and compare to the original robustness statistic.



Simulation in practice.

```
library(parallel)
# function to simulate response
sim_resp <- function(obj, ...){
  n <- nobs(obj)
  sim_e <- rnorm(n, 0, summary(obj)$sigma)
  X <- model.matrix(obj)
  sim_y <- X %*% coef(obj) + sim_e
  sim_y
}

# weights to get average predictions
wts <- pred_df$n/sum(pred_df$n)

# initialize new data
newdat <- ex_df
# set seed
set.seed(48104)
```

The next two blocks show one iteration, you'll have to wrap this in a loop or `lapply` to repeat.

Simulation Code

```
# simulate response
newdat$sim_y <- sim_resp(lmod)

# update models
lmod_up <- update(lmod, sim_y ~ ., data=newdat)
umod_up <- update(umod, sim_y ~ ., data=newdat)
mmod_up <- update(mmod, sim_y ~ ., data=newdat)

# Calculate predictions
leff_up <- predictions(lmod_up, by = "x", newdata = pred_df)
meff_up <- predictions(mmod_up, by = "x", newdata = pred_df)
ueff_up <- predictions(umod_up, by = "x", newdata = pred_df)

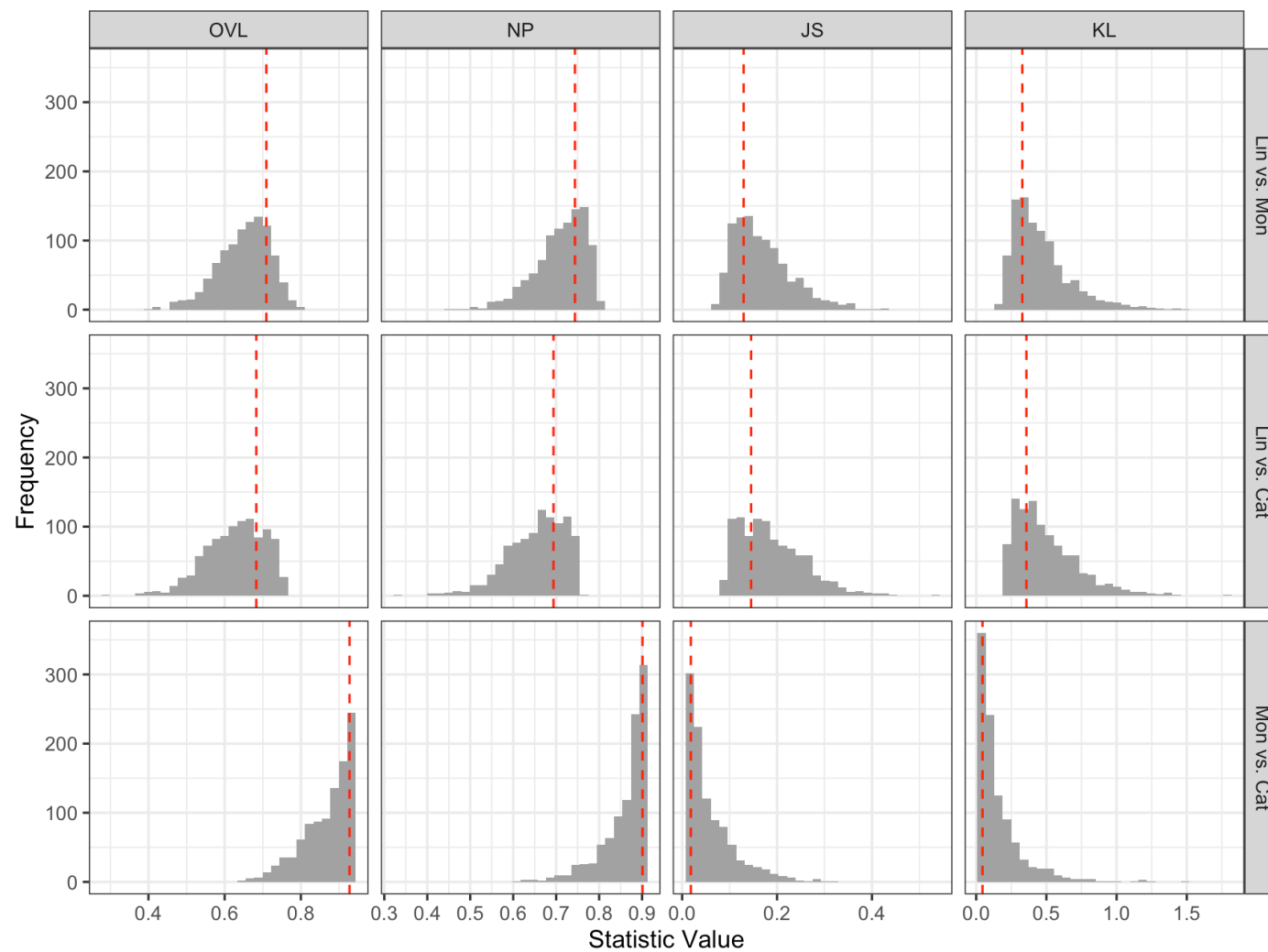
# Recover standard errors
sd_leff_up <- sqrt(diag(vcov(leff_up)))
sd_meff_up <- sqrt(diag(vcov(meff_up)))
sd_ueff_up <- sqrt(diag(vcov(ueff_up)))
```

```
# Calculate Robustness
rob_lm <- sapply(1:5, \(i)
  robustness(dnorm, dnorm,
    target_args = list(mean = coef(leff_up)[i],
                        sd = sd_leff_up[i]),
    alt_args = list(mean = coef(meff_up)[i],
                    sd = sd_meff_up[i])))

rob_mu <- sapply(1:5, \(i)
  robustness(dnorm, dnorm,
    target_args = list(mean = coef(meff_up)[i],
                        sd = sd_meff_up[i]),
    alt_args = list(mean = coef(ueff_up)[i],
                    sd = sd_ueff_up[i])))

# Make weighted average
c(c(rob_lm %*% wts), c(rob_lu %*% wts), c(rob_mu %*% wts))
```

Compare to Observed



Percentile Values

The numbers below are the percentile of the distribution in which the number falls.

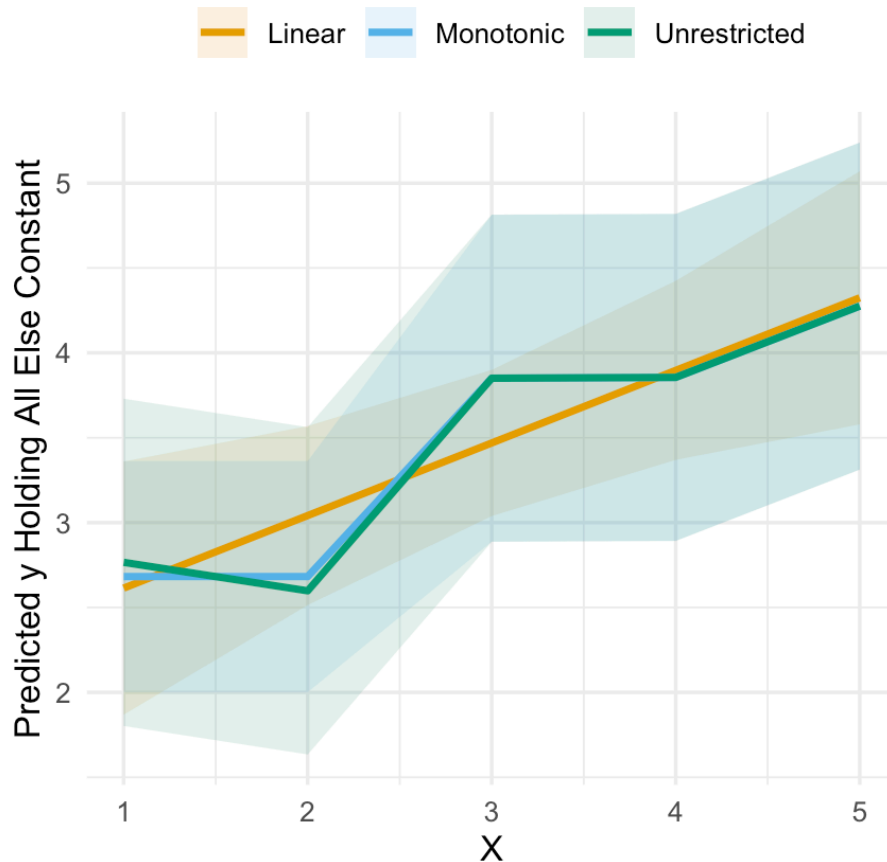
- for OVL and NP, it is the proportion below (less overlapping).
- for JS and KL, it is the proportion above (more divergent).

```
## # A tibble: 4 × 4
##   stat `Lin vs. Mon` `Lin vs. Cat` `Mon vs. Cat`
##   <fct>      <dbl>      <dbl>      <dbl>
## 1 OVL        0.805        0.722        0.824
## 2 NP         0.662        0.68         0.823
## 3 JS         0.696        0.684        0.823
## 4 KL         0.712        0.683        0.823
```

The observed robustness statistics are not unusually low relative to what would be expected if the linear model were correct, suggesting that any departures from linearity are modest.

- Any differences appear to arise from the equal-spacing rather than the monotonicity assumption.

Linear, Monoton and Unrestricted Effects



Functions overlap considerably.

- Increased efficiency with the linear function.
- Linear model is somewhat sensitive to relaxing equal spacing, but highly robust to relaxing monotonicity.
- None of the alternative effects indicates a substantively different story.

Exercise

We are going to replicate Ang et.al. (2019) from the *American Journal of Political Science*.

- They recode party ID (Strong D = 2, (D, Lean D) = 1, I=0, (R, Lean R) = -1, Strong R = -2)
 - `partyid7.may16` is the seven point variable that separates lean D and D as well as lean R and R.
- They estimate the effect of national economic evaluations on party ID among other vars.

```
load("ang_replication/tapsEcon.rData")

tapsData_T2 = tapsData %>% drop_na(male,income,white,black,hispanic,education,age2016,
                                   natecon.nov16.fd,natecon.jan17.fd,hhecon.nov16.fd,
                                   hhecon.jan17.fd,natcurr.nov16.fd,natcurr.jan17.fd,
                                   hhcurr.nov16.fd,hhcurr.jan17.fd)

base_mod <- lm((natcurr.jan17.fd) ~ partyid.may16.gh + male + income + white +
               black + hispanic + education + age2016,data=tapsData_T2)
```

Evaluate how robust the effect of party ID is to relevant assumptions.

Monotonicity in GLMs

For single equation models (binary, poisson, gamma, ordered), do the following:

1. Create the unrestricted model with the appropriate GLM family.
2. Save the model predictions on the link scale back into the data as `yhat_link`.
3. Estimate a linear model of `yhat_link` on the unrestricted model formula.
4. Do `make_mono_var()` on the model from 3 to create the variable of interest with monotonicity constraints.

```
library(foreign)
anes <- rio::import("../data/anes1992.dta")
anesu <- anes %>% mutate(pid_fac = as.factor(pid)) |>
  drop_na(votedem, retnat, pid, age, male,
          educ, black, south)
linmod <- glm(votedem ~ retnat + pid + age + male + educ +
             black + south, data=anesu, family=binomial)

catmod <- glm(votedem ~ retnat + pid_fac + age + male +
             educ + black + south, data=anesu, family=binomial)

anesu$yhat_link <- predict(catmod, type="link")
auxmod <- lm(yhat_link ~ retnat + pid_fac + age + male +
            educ + black + south, data=anesu)
anesu <- make_mono_var(auxmod, anesu, mono_var = "pid_fac", d
pid_map <- anesu |> select(pid, pid_fac_ord) |> distinct() |>
monmod <- glm(votedem ~ retnat + age + male + educ + black +
             recode_values(pid, from=pid_map$pid, to=pid_map$pid
             data=anesu, family=binomial)
```

Robustness with GLMs

The main differences with non-Gaussian GLMs are two-fold.

1. The object of robustness analysis should generally be predicted probabilities, expected values, or first differences - quantities expressed on the observed outcome scale and thus comparable across specifications.
2. The sampling distributions of these quantities are often not available in closed form or are poorly approximated by normal distributions. Consequently, robustness calculations are typically based on simulation, asymptotic approximations, or quasi-posterior draws rather than analytic probability distributions.

Robustness with GLMs

The main differences with non-Gaussian GLMs are two-fold.

1. The object of robustness analysis should generally be predicted probabilities, expected values, or first differences - quantities expressed on the observed outcome scale and thus comparable across specifications.
2. The sampling distributions of these quantities are often not available in closed form or are poorly approximated by normal distributions. Consequently, robustness calculations are typically based on simulation, asymptotic approximations, or quasi-posterior draws rather than analytic probability distributions.

The steps are the same as before, but use `avg_predictions()` or `avg_comparisons()` instead of `predictions()` as above.

Predictions or First Differences?

Predictions:

- Bounded in $(0,1)$
- Near 0 or 1, uncertainty is often asymmetric and poorly approximated by a normal distribution.
- Predicted probabilities can frequently approach these boundaries (e.g., rare events).
- Robustness calculations are often best based on quasi-posterior draws.

Exception: When predictions are well away from the boundaries and uncertainty is small, normal approximations may be adequate for rapid evaluation.

Predictions or First Differences?

Predictions:

- Bounded in $(0,1)$
- Near 0 or 1, uncertainty is often asymmetric and poorly approximated by a normal distribution.
- Predicted probabilities can frequently approach these boundaries (e.g., rare events).
- Robustness calculations are often best based on quasi-posterior draws.

Exception: When predictions are well away from the boundaries and uncertainty is small, normal approximations may be adequate for rapid evaluation.

First Differences:

- Bounded in $(-1,1)$
- Often lie well away from their boundaries, even when the underlying predictions do not.
- Uncertainty is frequently better approximated by a normal distribution.
- Delta-method standard errors are often adequate for robustness calculations, simulations can always be used for greater accuracy.

Predictions or First Differences?

Predictions:

- Bounded in $(0,1)$
- Near 0 or 1, uncertainty is often asymmetric and poorly approximated by a normal distribution.
- Predicted probabilities can frequently approach these boundaries (e.g., rare events).
- Robustness calculations are often best based on quasi-posterior draws.

Exception: When predictions are well away from the boundaries and uncertainty is small, normal approximations may be adequate for rapid evaluation.

First Differences:

- Bounded in $(-1,1)$
- Often lie well away from their boundaries, even when the underlying predictions do not.
- Uncertainty is frequently better approximated by a normal distribution.
- Delta-method standard errors are often adequate for robustness calculations, simulations can always be used for greater accuracy.



Robustness of Predictions

```
# predictions from linear model
linpreds <- avg_predictions(
  linmod,
  variables = list(pid=1:7),
  by="pid") |>
  inferences(method="simulation", R=5000)
lin_post <- get_draws(linpreds)

# predictions from monotonic model
monpreds <- avg_predictions(
  monmod,
  variables = list(pid=1:7),
  by="pid") |>
  inferences(method="simulation", R=5000)
mon_post <- get_draws(monpreds)

# predictions from unrestricted model
catpreds <- avg_predictions(
  catmod,
  variables = "pid_fac",
  by="pid_fac") |>
  inferences(method="simulation", R=5000)
cat_post <- get_draws(catpreds)
```

```
lp_mat <- lin_post |> select(drawid, pid, draw) |>
  pivot_wider(names_from = "pid", names_prefix="pid_",
    values_from = "draw")
mp_mat <- mon_post |> select(drawid, pid, draw) |>
  pivot_wider(names_from = "pid", names_prefix="pid_",
    values_from = "draw")
cp_mat <- cat_post |> select(drawid, pid_fac, draw) |>
  pivot_wider(names_from = "pid_fac", names_prefix="pid_",
    values_from = "draw")
rob_lm <- sapply(paste0("pid_", 1:7), \(x){
  robustness(lp_mat[[x]], mp_mat[[x]])
})
rob_lu <- sapply(paste0("pid_", 1:7), \(x){
  robustness(lp_mat[[x]], cp_mat[[x]])
})
rob_mu <- sapply(paste0("pid_", 1:7), \(x){
  robustness(mp_mat[[x]], cp_mat[[x]])
})
```



Robustness Values

```
# relax equal-spacing assumption
as_tibble(rob_lm)
```

```
## # A tibble: 4 × 7
##   pid_1 pid_2 pid_3 pid_4 pid_5 pid_6 pid_7
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 0.459 0.0711 0.0452 0.567 0.0724 0.456 0.601
## 2 0.496 0.0455 0.0196 0.535 0.0441 0.599 0.597
## 3 0.312 0.864 0.909 0.224 0.858 0.315 0.191
## 4 0.807 8.50 8.81 0.539 5.48 0.948 0.462
```

```
# relax monotonicity assumption
as_tibble(rob_mu)
```

```
## # A tibble: 4 × 7
##   pid_1 pid_2 pid_3 pid_4 pid_5 pid_6 pid_7
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 0.966 0.697 0.692 0.984 0.605 0.662 0.981
## 2 0.942 0.765 0.751 0.947 0.669 0.764 0.928
## 3 0.00139 0.108 0.114 0.000616 0.177 0.135 0.00215
## 4 0.00397 0.269 0.279 0.00247 0.445 0.349 0.00501
```

```
# overall difference
as_tibble(rob_lu)
```

```
## # A tibble: 4 × 7
##   pid_1 pid_2 pid_3 pid_4 pid_5 pid_6 pid_7
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 0.466 0.0641 0.0387 0.561 0.0587 0.253 0.603
## 2 0.496 0.0388 0.0172 0.528 0.0336 0.283 0.597
## 3 0.305 0.875 0.919 0.228 0.879 0.570 0.191
## 4 0.776 5.02 5.32 0.549 4.38 1.93 0.460
```

- Linear is not particularly robust to monotone, especially in the middle
- Monotone is quite robust to non-monotone
- The overall difference comes largely from lack of robustness to the equal spacing assumption.

Simulation Length

When using simulated values, people may wonder - *How many simulations do I need?*

Defaults:

- Quick comparison - 2000 draws
- Reported in paper - 10000 + draws

Simulation Length

When using simulated values, people may wonder - *How many simulations do I need?*

Defaults:

- Quick comparison - 2000 draws
- Reported in paper - 10000 + draws

More simulations are needed when trying to characterize quantities in the tail of a distribution.

- Further out in the tail → more draws.
- Quantities based on extreme percentiles generally require more draws than quantities based on means or medians.

Simulation Length

When using simulated values, people may wonder - *How many simulations do I need?*

Defaults:

- Quick comparison - 2000 draws
- Reported in paper - 10000 + draws

More simulations are needed when trying to characterize quantities in the tail of a distribution.

- Further out in the tail → more draws.
- Quantities based on extreme percentiles generally require more draws than quantities based on means or medians.

Speeding up `inferences(method="simulation")`

- Use "average" respondent rather than average effects/predictions.



Robustness of First Differences.

```
lin_fd <- avg_predictions(  
  linmod,  
  variables = list(pid=c(1,3,4,6,7)),  
  hypothesis = ~sequential)  
  
mon_fd <- avg_predictions(  
  monmod,  
  variables = list(pid=c(1,3,4,6,7)),  
  hypothesis = ~sequential)  
  
cat_fd <- avg_predictions(  
  catmod,  
  variables = list(pid_fac=c("1", "3", "4", "6", "7")),  
  hypothesis = ~sequential)  
  
se_linfd <- sqrt(diag(vcov(lin_fd)))  
se_monfd <- sqrt(diag(vcov(mon_fd)))  
se_catfd <- sqrt(diag(vcov(cat_fd)))
```

Contrasts are chosen to avoid transitions entirely within pooled monotone categories.

```
robfd_lm <- sapply(1:4, \(i)  
  robustness(dnorm, dnorm,  
    target_args = list(mean = coef(lin_fd)[i],  
                        sd = se_linfd[i]),  
    alt_args = list(mean = coef(mon_fd)[i],  
                    sd = se_monfd[i])))  
  
robfd_mu <- sapply(1:4, \(i)  
  robustness(dnorm, dnorm,  
    target_args = list(mean = coef(mon_fd)[i],  
                        sd = se_monfd[i]),  
    alt_args = list(mean = coef(cat_fd)[i],  
                    sd = se_catfd[i])))  
  
robfd_lu <- sapply(1:4, \(i)  
  robustness(dnorm, dnorm,  
    target_args = list(mean = coef(lin_fd)[i],  
                        sd = se_linfd[i]),  
    alt_args = list(mean = coef(cat_fd)[i],  
                    sd = se_catfd[i])))  
  
colnames(robfd_lm) <- colnames(robfd_mu) <-  
  colnames(robfd_lu) <- c("1 -> 3", "3 -> 4",  
                           "4 -> 6", "6 -> 7")
```



FD Robustness Values

```
# equal-spacing  
as_tibble(robfd_lm, rownames="stat")
```

```
## # A tibble: 4 × 5  
##   stat `1 -> 3` `3 -> 4` `4 -> 6` `6 -> 7`  
##   <chr>    <dbl>    <dbl>    <dbl>    <dbl>  
## 1 ovl      0.134      0.148      0.384      0.145  
## 2 np       0.108      0.112      0.339      0.108  
## 3 js       0.734      0.704      0.403      0.708  
## 4 kl       2.51       2.20       1.03       2.21
```

```
# monotonicity  
as_tibble(robfd_mu, rownames="stat")
```

```
## # A tibble: 4 × 5  
##   stat `1 -> 3` `3 -> 4` `4 -> 6` `6 -> 7`  
##   <chr>    <dbl>    <dbl>    <dbl>    <dbl>  
## 1 ovl      0.761      0.857      0.850      0.714  
## 2 np       0.839      0.914      0.917      0.815  
## 3 js       0.0691     0.0240     0.0259     0.0950  
## 4 kl       0.173      0.0631     0.0693     0.241
```

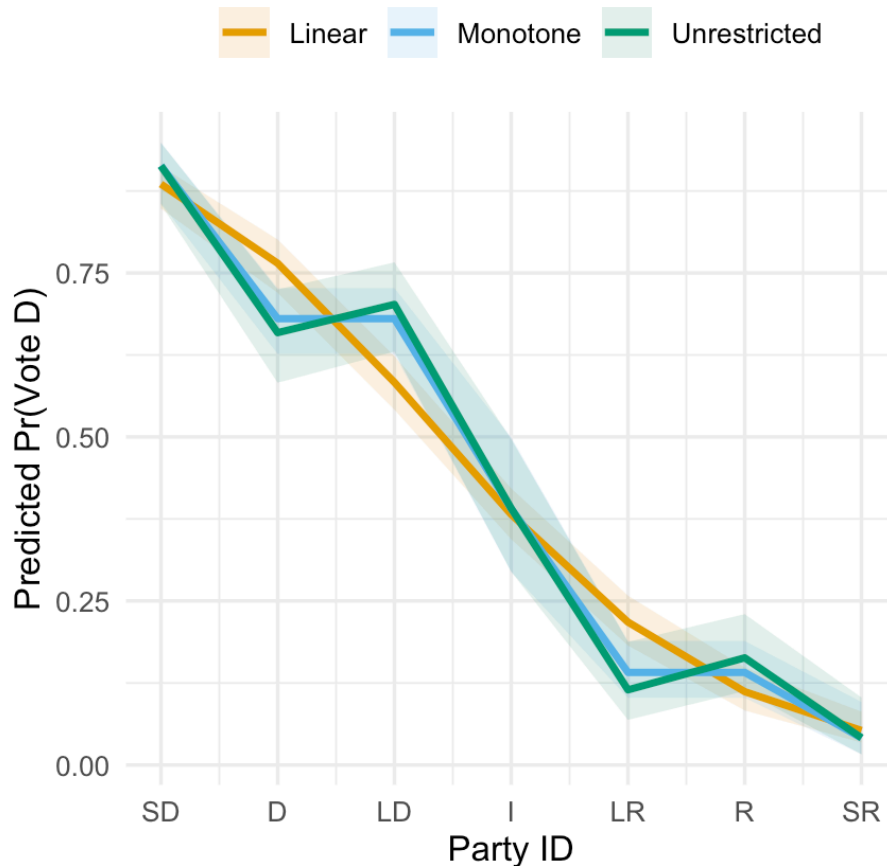
```
# overall difference  
as_tibble(robfd_lu, rownames="stat")
```

```
## # A tibble: 4 × 5  
##   stat `1 -> 3` `3 -> 4` `4 -> 6` `6 -> 7`  
##   <chr>    <dbl>    <dbl>    <dbl>    <dbl>  
## 1 ovl      0.0894     0.103     0.317     0.0745  
## 2 np       0.0631     0.0729     0.270     0.0496  
## 3 js       0.808      0.777     0.479     0.827  
## 4 kl       2.97       2.63      1.26      3.00
```

Same conclusion as above

- Not robust to equal spacing assumption
- Quite robust to monotonicity assumption.

Effects



Robustness Conclusions

- Inferential robustness is low to relaxing the equal spacing assumption.
- Main problem is what happens between partisans and leaners
- Substantive story - moving from Strong Democratic to Independent and Independent to Strong Republican tells a very similar story. Substantive conclusions are generally robust.
- Predictions and first differences tell the same story.

Exercise

Banda and Casesse have a paper in *Political Behavior* (2021) about sexism, racial resentment, party ID and voter turnout. One of their control variables is retrospective economic evaluations (`econanx`). This is treated linearly in their model. - What happens if we relax the relevant assumptions on economic anxiety?

- Is the effect of economic anxiety robust?
- Their main arguments are about the slopes of sexism and resentment by `pid3`. Do those survive?

Starter code on next slide

Banda and Casesse Replication Starter Code

```
bc <- rio::import("banda_casesse_rep/banda_casesse_data_postprocess.dta")

bc <- bc |>
  select("vote2016_clerical", "pid3", "sexism", "pid3", "resentment", "ideo7", "female",
"interest", "edu", "income", "cattend", "age", "econanx", "competition",
"race5", "V160102", "V160202") |>
  drop_na() |>
  mutate(pid3 = rio::factorize(pid3),
         econanx_fac = rio::factorize(econanx),
         econanx_facn = as.factor(econanx)) |>
  filter(race5 == 1)

library(survey)
bc_des <- svydesign(id=~1, psu=~V160202, weights=~V160102, data=bc)

bcmo_l <- svyglm(vote2016_clerical ~ pid3*sexism + pid3*resentment + ideo7 +
               female + interest + edu+ income + cattend + age + econanx +
               competition, design=bc_des, family=binomial)

avg_slopes(bcmo_l, variables="resentment", by="pid3")
avg_slopes(bcmo_l, variables="sexism", by="pid3")
```




Functional Form Robustness Workflow

1. Identify the assumptions implied by the target model.
2. Relax one assumption at a time.
3. Evaluate robustness using predictions or first differences.
4. Determine which assumptions matter.
5. Describe the alternative relationships when robustness is low.